Sorting

- arranging elements of set into order

- Solution:
  - Insertion Sort
  - Quicksort
  - Mergesort
  - Heapsort
  - Shellsort
  - Radix Sorting

Application of Sorting

- For searching on unsorted data by comparing keys, optimal solutions require □(n) comparisons.
- For searching on sorted data by comparing keys, optimal solutions require □(log n) comparisons.
- Sorting data for users
- More…

- **Comparison based sorting**:. (eg. insertion sort. merge sort, bubble sort, quick sort etc.
- **Non-Comparison based sorting**: (eg. bucket sort, radix sort, counting sort etc.)
- **Internal Sorting**: (eg. bubble sort. insertion sort. etc.)
- **External Sorting**: (eg. quick sort. merge sort. etc.)
- **Stable sorting**: (eg. merge sort, insertion sort, bubble sort)
- **Unstable sorting** :(quicksort, heap sort, and selection sort)
- **Adaptive Sorting** : (i.e. sorted or unsorted data eg. bubble sort, insertion sort, quick sort etc)
- **Non-Adaptive Sorting** : (eg. selection sort, merge sort, heap sort etc.)
- **In-place/Outplace technique** – (eg. in comparison based techniques mentioned above only merge sort is outplaced. Among the non-comparison based techniques mentioned above all are outplaced techniques.

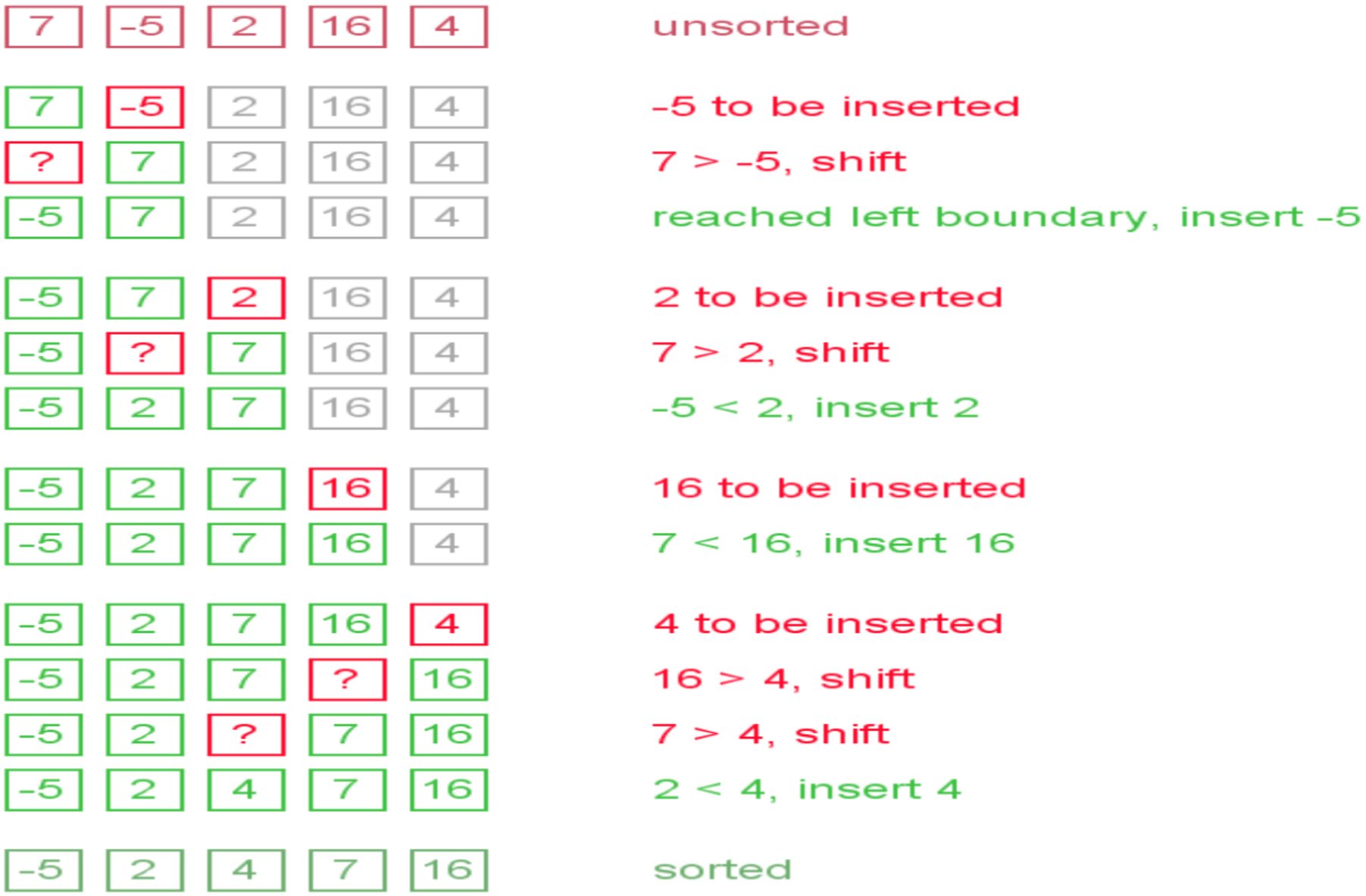
Insertion Sort



Insertion Sort

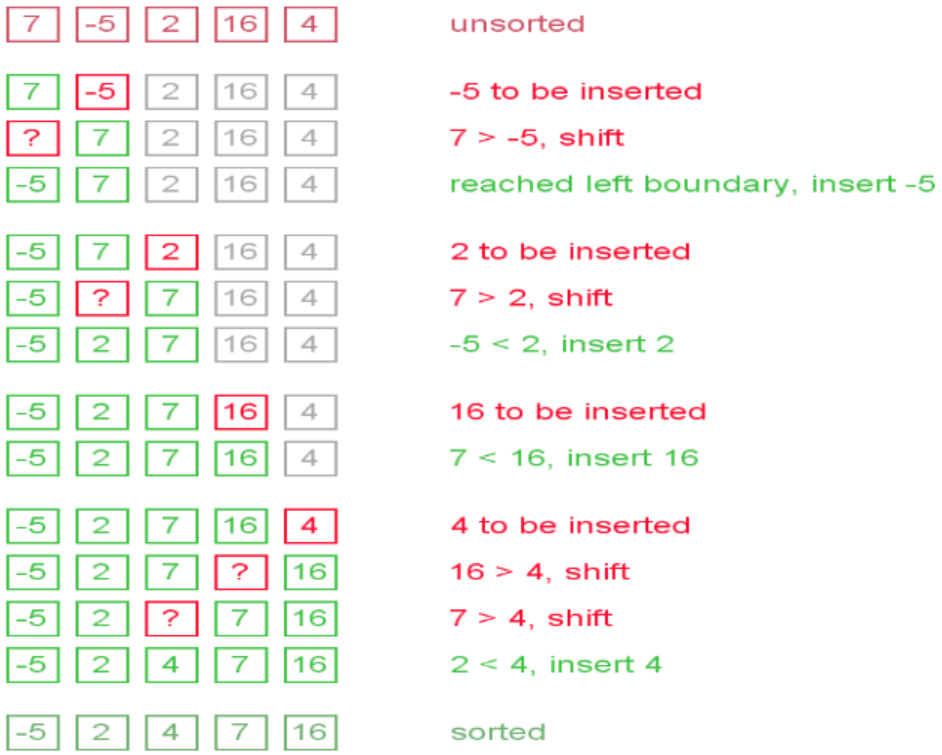
- Strategy: Insertion of an element in proper order:
  - Begin with a sequence E of n elements in arbitrary order
  - Initially assume the sorted segment contains first element
  - Let x be the next element to be inserted in sorted segment, pull x “out of the way”, leaving a vacancy
  - repeatedly compare x to the element just to the left of the vacancy, and as long as x is smaller, move that element into the vacancy,
  - else put x in the vacancy,
  - repeat the next element that has not yet examined.

Insertion Sort



Insertion Sort (A)

```
for j=2 to A.length
    key = A[j]
    //insert A[j] into sorted sequence A[1..j-1]
    i=j-1
   while (i>0 and A[i] > key )
      A[i+1] =A [i]
       i=i-1
 A[i+1]= key
```



Insertion Sort: Analysis

- Worst-Case Complexity
  - $W(n) = O(n^2)$
  - When the elements are in reversed order

- Best Case
  - W(n) = Omega(n)
  - When the elements are already sorted in ascending order

Analysis [no of comparisons/movement]

$F(n) = 1 + 2 + 3 + \ldots\ldots\ldots(n-3) + (n-2) + (n-1)$
$= n(n-1)/2$
$= O(n^2)$

Improvement Analysis

(by reducing number of ***comparisons*** and ***movement***)

Reduce number of Comparison

- Apply binary search

Reduce number of Movement

- -Implement doubly linked list

- cannot make total running time better than $O(n^2)$

Video Lecture of the topic can be found at

Insertion Sort Analysis

https://youtu.be/jikkNiuR2CQ